

Practical-5

Aim:

To install and configure MongoDB and perform **CRUD (Create, Read, Update, Delete) operations** on MongoDB collections, along with aggregation, indexing, query performance comparison, JSON data import, and analytical queries.

Problem Definition:

Perform Create, Read, Update, and Delete (CRUD) operations using MongoDB. Students will work with a NoSQL document database to understand how unstructured and semi-structured data is managed at scale in Big Data environments.

Objective:

1. To understand the basic concepts of MongoDB and NoSQL databases.
2. To create a MongoDB database and collection.
3. To perform Create, Read, Update, and Delete (CRUD) operations.
4. To insert single and multiple documents into a collection.
5. To retrieve and filter documents using MongoDB queries.
6. To update and delete existing documents.
7. To use the aggregation pipeline for data analysis and summary reports.
8. To create indexes and compare query performance with and without indexes.
9. To import a JSON dataset into MongoDB.
10. To perform analytical queries on the imported dataset.

Objective/Output:

Tasks: 1. Install and configure MongoDB.

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\jttha>mongosh --version
2.11.1

C:\Users\jttha>mongod --version
'mongod' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\jttha>Get-Service MongoDB
'Get-Service' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\jttha>mongosh
Current Mongosh Log ID: 6aaad072a6c3b83d8d864540
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.11.1
Using MongoDB:      8.3.11
Using Mongosh:       2.11.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only, and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-09-16T17:48:09.415+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----
```

2. Create a database and collection.

```
test> use CollegeDB
switched to db CollegeDB
CollegeDB> db
CollegeDB
CollegeDB> db.createCollection("students")
{ ok: 1 }
CollegeDB> show collections
students
```

3. Insert documents (single and multiple).

```
CollegeDB> db.students.insertOne({
  rollNo: 101,
  name: "Rahul",
  age: 20,
  department: "IT",
  semester: 5,
  marks: 82,
  city: "Ahmedabad"
})
{
  acknowledged: true,
  insertedId: ObjectId('6aaad0c1a6c3b83d8d864541')
}
CollegeDB> db.students.insertMany([
  {
    rollNo: 102,
    name: "Priya",
    age: 21,
    department: "CSE",
    semester: 5,
    marks: 91,
    city: "Vadodara"
  },
  {
    rollNo: 103,
    name: "Amit",
    age: 20,
    department: "IT",
    semester: 5,
    marks: 76,
    city: "Surat"
  }
])
```

```
{
  {
    rollNo: 105,
    name: "Karan",
    age: 22,
    department: "CE",
    semester: 6,
    marks: 69,
    city: "Rajkot"
  },
  {
    rollNo: 106,
    name: "Anjali",
    age: 20,
    department: "IT",
    semester: 5,
    marks: 95,
    city: "Vadodara"
  }
]
}
acknowledged: true,
insertedIds: {
  '0': ObjectId('6aaad0d1a6c3b83d8d864542'),
  '1': ObjectId('6aaad0d1a6c3b83d8d864543'),
  '2': ObjectId('6aaad0d1a6c3b83d8d864544'),
  '3': ObjectId('6aaad0d1a6c3b83d8d864545'),
  '4': ObjectId('6aaad0d1a6c3b83d8d864546')
}
}
```

4. Read/Query documents using various filters.

```
CollegeDB> db.students.find()
[
  {
    _id: ObjectId('6aaad0c1a6c3b83d8d864541'),
    rollNo: 101,
    name: 'Rahul',
    age: 20,
    department: 'IT',
    semester: 5,
    marks: 82,
    city: 'Ahmedabad'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864542'),
    rollNo: 102,
    name: 'Priya',
    age: 21,
    department: 'CSE',
    semester: 5,
    marks: 91,
    city: 'Vadodara'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864543'),
    rollNo: 103,
    name: 'Amit',
    age: 20,
    department: 'IT',
    semester: 5,
    marks: 76,
    city: 'Surat'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864544'),
    rollNo: 104,
    name: 'Neha',
    age: 21,
    department: 'CSE',
    semester: 5,
    marks: 88,
    city: 'Ahmedabad'
  }
]
```

```
CollegeDB> db.students.findOne({ rollNo: 101 })
{
  _id: ObjectId('6aaad0c1a6c3b83d8d864541'),
  rollNo: 101,
  name: 'Rahul',
  age: 20,
  department: 'IT',
  semester: 5,
  marks: 82,
  city: 'Ahmedabad'
}
CollegeDB> db.students.find({ department: "IT" })
[
  {
    _id: ObjectId('6aaad0c1a6c3b83d8d864541'),
    rollNo: 101,
    name: 'Rahul',
    age: 20,
    department: 'IT',
    semester: 5,
    marks: 82,
    city: 'Ahmedabad'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864543'),
    rollNo: 103,
    name: 'Amit',
    age: 20,
    department: 'IT',
    semester: 5,
    marks: 76,
    city: 'Surat'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864546'),
    rollNo: 106,
    name: 'Anjali',
    age: 20,
    department: 'IT',
    semester: 5,
    marks: 95,
  }
]
```

```
CollegeDB> db.students.find({ marks: { $gt: 80 } })
[
  {
    _id: ObjectId('6aaad0c1a6c3b83d8d864541'),
    rollNo: 101,
    name: 'Rahul',
    age: 20,
    department: 'IT',
    semester: 5,
    marks: 82,
    city: 'Ahmedabad'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864542'),
    rollNo: 102,
    name: 'Priya',
    age: 21,
    department: 'CSE',
    semester: 5,
    marks: 91,
    city: 'Vadodara'
  },
]
```

5. Update existing documents.

```
CollegeDB> db.students.updateOne(
|   { rollNo: 101 },
|   { $set: { marks: 87 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
CollegeDB> db.students.findOne({ rollNo: 101 })
{
  _id: ObjectId('6aaad0c1a6c3b83d8d864541'),
  rollNo: 101,
  name: 'Rahul',
  age: 20,
  department: 'IT',
  semester: 5,
  marks: 87,
  city: 'Ahmedabad'
}
CollegeDB> db.students.updateMany(
|   { department: "IT" },
|   { $set: { department: "Information Technology" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
CollegeDB> db.students.find()
[
  {
    _id: ObjectId('6aaad0c1a6c3b83d8d864541'),
    rollNo: 101,
    name: 'Rahul',
    age: 20,
    department: 'Information Technology',
```

6. Delete documents and collections.

```
CollegeDB> db.students.deleteOne({ rollNo: 105 })
{ acknowledged: true, deletedCount: 1 }
CollegeDB> db.students.find()
[
  {
    _id: ObjectId('6aaad0c1a6c3b83d8d864541'),
    rollNo: 101,
    name: 'Rahul',
    age: 20,
    department: 'Information Technology',
    semester: 5,
    marks: 92,
    city: 'Ahmedabad'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864542'),
    rollNo: 102,
    name: 'Priya',
    age: 21,
    department: 'CSE',
    semester: 5,
    marks: 91,
    city: 'Vadodara'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864543'),
    rollNo: 103,
    name: 'Amit',
    age: 20,
    department: 'Information Technology',
    semester: 5,
    marks: 76,
    city: 'Surat'
  },
  {
    _id: ObjectId('6aaad0d1a6c3b83d8d864544'),
    rollNo: 104,
    name: 'Neha',
    age: 21,
    department: 'CSE',
  }
]
```

7. Use MongoDB aggregation pipeline for data analysis.

```

CollegeDB> db.students.aggregate([
|   {
|     $group: {
|       _id: "$department",
|       averageMarks: { $avg: "$marks" }
|     }
|   }
| ])
| [
|   { _id: 'Information Technology', averageMarks: 87.66666666666667 },
|   { _id: 'CSE', averageMarks: 89.5 }
| ]
CollegeDB> db.students.aggregate([
|   {
|     $group: {
|       _id: "$department",
|       totalStudents: { $sum: 1 }
|     }
|   }
| ])
| [
|   { _id: 'CSE', totalStudents: 2 },
|   { _id: 'Information Technology', totalStudents: 3 }
| ]
CollegeDB> db.students.aggregate([
|   {
|     $group: {
|       _id: null,
|       highestMarks: { $max: "$marks" }
|     }
|   }
| ])
| [ { _id: null, highestMarks: 95 } ]
CollegeDB> db.students.aggregate([
|   {
|     $group: {
|       _id: null,
|       lowestMarks: { $min: "$marks" }
|     }
|   }
| ])
| [ { _id: null, lowestMarks: 75 } ]

```

```

CollegeDB> db.students.aggregate([
|   {
|     $group: {
|       _id: "$department",
|       averageMarks: { $avg: "$marks" }
|     }
|   },
|   {
|     $sort: {
|       averageMarks: -1
|     }
|   }
| ])
| [
|   { _id: 'CSE', averageMarks: 89.5 },
|   { _id: 'Information Technology', averageMarks: 87.66666666666667 }
| ]

```

Key Questions /Analysis:

Q1. What is MongoDB and how does it differ from relational databases?

Difference between MongoDB and Relational Databases

MongoDB	Relational Database
NoSQL database	SQL database
Stores data as documents	Stores data in tables
Uses BSON documents	Uses rows and columns
Schema is flexible	Schema is generally fixed
Supports horizontal scaling	Traditionally focuses on vertical scaling
Examples: MongoDB	Examples: MySQL, PostgreSQL, Oracle

Example MongoDB document:

```
{  
  "rollNo": 101,  
  "name": "Rahul",  
  "department": "IT",  
  "marks": 82  
}
```

Q2. What are CRUD operations and why are they fundamental to database management?

CRUD stands for:

- **C – Create:** Insert new data into the database.
- **R – Read:** Retrieve or view existing data.
- **U – Update:** Modify existing data.
- **D – Delete:** Remove data from the database.

MongoDB examples:**Create:**

```
db.students.insertOne({  
  rollNo: 101,  
  name: "Rahul",  
  marks: 82  
})
```

Read:

```
db.students.find()
```

Update:

```
db.students.updateOne(  
  {rollNo: 101},  
  {$set: {marks: 90}}  
)
```

Delete:

```
db.students.deleteOne({rollNo: 101})
```

CRUD operations are fundamental because almost every database application needs to **store, retrieve, modify, and remove data**.

Q3. How does MongoDB store data (BSON format)?

MongoDB stores data as **BSON (Binary JSON)** documents.

BSON is similar to JSON but is stored in a **binary representation** and supports additional data types such as dates, integers, decimal values, and binary data.

For example:

```
{  
  "rollNo": 101,  
  "name": "Rahul",
```

```
"marks": 82,  
"department": "IT"  
}
```

MongoDB stores this information internally as a BSON document.

Advantages of BSON:

1. Supports more data types than JSON.
2. Efficient for storing and processing data.
3. Allows documents to contain nested objects and arrays.
4. Suitable for MongoDB's document-oriented structure.

Q4. What is the aggregation pipeline in MongoDB?

The **aggregation pipeline** is a framework in MongoDB used to **process, transform, and analyze data** through a sequence of stages.

Each stage performs a particular operation on the data, and the output of one stage becomes the input of the next stage.

Common aggregation stages:

- \$match – filters documents.
- \$group – groups documents.
- \$sort – sorts documents.
- \$project – selects or modifies fields.
- \$limit – limits the number of documents.

Example:

```
db.students.aggregate([  
  {  
    $group: {  
      _id: "$department",  
      averageMarks: {$avg: "$marks"},  
    }  
  }  
])
```

```
        totalStudents: {$sum: 1}
      }
    },
    {
      $sort: {
        averageMarks: -1
      }
    }
  ]
}
```

This example groups students according to their **department**, calculates the **average marks** and **number of students**, and then sorts the result by average marks.

Q5. When should NoSQL databases be preferred over SQL databases?

NoSQL databases can be preferred when an application requires **flexible schemas, large-scale data storage, high scalability, or handling rapidly changing data structures**.

NoSQL is useful when:

1. **Data structure changes frequently**
Different documents can have different fields.
2. **Large volumes of data are handled**
NoSQL databases can be designed to scale horizontally across multiple servers.
3. **High traffic and scalability are required**
They are commonly used for applications with large numbers of users or requests.
4. **Unstructured or semi-structured data is involved**
Examples include JSON-like application data, logs, and some IoT data.

5. Fast development is important

A flexible schema can make it easier to evolve an application.

SQL databases are often preferred when:

- Data has strong relationships.
- Complex joins are central to the application.
- Strict consistency and structured schemas are important.
- The application depends heavily on transactions.

Supplementary Problems:

Create an index on a collection and compare query performance. Use the aggregation pipeline to generate a summary report. Import a JSON dataset into MongoDB and perform analytical queries.

```
CollegeDB> db.students.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
CollegeDB> db.students.createIndex({ rollNo: 1 })
rollNo_1
CollegeDB> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { rollNo: 1 }, name: 'rollNo_1' }
]
CollegeDB> db.students.find({ rollNo: 103 }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'CollegeDB.students',
    parsedQuery: { rollNo: { '$eq': 103 } },
    indexFilterSet: false,
    queryHash: '0DB48064',
    planCacheShapeHash: '0DB48064',
    planCacheKey: 'FEC0A1B5',
    optimizationTimeMillis: 8,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      nss: 'CollegeDB.students',
      inputStage: {
        stage: 'IXSCAN',
        nss: 'CollegeDB.students',
        keyPattern: { rollNo: 1 },
        indexName: 'rollNo_1',
        isMultiKey: false,
        multiKeyPaths: { rollNo: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
```

```
C:\Users\jttha>"C:\Users\jttha\Downloads\mongodb-database-tools-windows-x86_64-100.18.0\mongodb-database-tools-windows-x86_64-100.18.0\bin\mongoimport.exe"
--db CollegeDB --collection studentsJSON --file "C:\Users\jttha\Downloads\students.json" --jsonArray
2026-09-16T23:18:34.334+0530    connected to: mongodb://localhost/
2026-09-16T23:18:34.366+0530    8 document(s) imported successfully. 0 document(s) failed to import.

C:\Users\jttha>mongosh
Current Mongosh Log ID: 6aaad6962d7d4d899f3ae99f
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.11.1
Using MongoDB:      8.3.11
Using Mongosh:      2.11.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-09-16T17:48:09.415+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  -----

test> use CollegeDB
switched to db CollegeDB
CollegeDB> db.studentsJSON.find().pretty()
[
  {
    _id: ObjectId('6aaad67266df39e8489a3844'),
    rollNo: 102,
    name: 'Priya',
    department: 'CSE',
    semester: 5,
    marks: 91,
    city: 'Vadodara'
  },
  {
    _id: ObjectId('6aaad67266df39e8489a3845'),
    rollNo: 101,
    name: 'Rahul',
    department: 'IT',
    semester: 5,
    marks: 82,
    city: 'Ahmedabad'
  },
]
```

```
CollegeDB> db.studentsJSON.find().sort({marks: -1}).limit(3).pretty()
[
  {
    _id: ObjectId('6aaad67266df39e8489a3847'),
    rollNo: 106,
    name: 'Anjali',
    department: 'IT',
    semester: 5,
    marks: 95,
    city: 'Vadodara'
  },
  {
    _id: ObjectId('6aaad67266df39e8489a3844'),
    rollNo: 102,
    name: 'Priya',
    department: 'CSE',
    semester: 5,
    marks: 91,
    city: 'Vadodara'
  },
  {
    _id: ObjectId('6aaad67266df39e8489a3849'),
    rollNo: 108,
    name: 'Pooja',
    department: 'IT',
    semester: 6,
    marks: 89,
    city: 'Ahmedabad'
  }
]
```

```
CollegeDB> db.studentsJSON.countDocuments()
8
CollegeDB> db.studentsJSON.countDocuments({department: "IT"})
4
CollegeDB> db.studentsJSON.aggregate([
|   {
|     $group: {
|       _id: "$department",
|       totalStudents: {$sum: 1},
|       averageMarks: {$avg: "$marks"},
|       highestMarks: {$max: "$marks"},
|       lowestMarks: {$min: "$marks"}
|     }
|   },
|   {
|     $sort: {
|       averageMarks: -1
|     }
|   }
| ])
[
| {
|   _id: 'IT',
|   totalStudents: 4,
|   averageMarks: 85.5,
|   highestMarks: 95,
|   lowestMarks: 76
| },
| {
|   _id: 'CSE',
|   totalStudents: 3,
|   averageMarks: 84,
|   highestMarks: 91,
|   lowestMarks: 73
| },
| {
|   _id: 'CE',
|   totalStudents: 1,
|   averageMarks: 69,
|   highestMarks: 69,
|   lowestMarks: 69
| }
| ]
```

Tools and Technologies:

- **Database:** MongoDB 8.3.11
- **Database Type:** NoSQL / Document-Oriented Database
- **Database Shell:** MongoDB Shell (mongosh)
- **Database Tools:** MongoDB Database Tools
- **Data Format:** BSON / JSON

- **Import Tool:** mongoimport
- **GUI Tool:** MongoDB Compass
- **Operating System:** Windows
- **Command-Line Environment:** Windows Command Prompt (CMD)
- **Query Language:** MongoDB Query Language (MQL)
- **Data Processing:** MongoDB Aggregation Pipeline
- **Indexing:** MongoDB Indexes
- **Dataset:** Student JSON Dataset

Conclusion:

The practical was successfully completed using MongoDB. We learned how MongoDB stores data as flexible BSON documents and performed all major CRUD operations on collections. We also imported a JSON dataset using mongoimport, performed analytical queries, and generated a summary report using the aggregation pipeline. An index was created and query execution was compared with and without indexing. Thus, this practical provided hands-on knowledge of MongoDB, NoSQL databases, CRUD operations, aggregation, indexing, JSON data import, and data analysis.

Github Repo:

https://github.com/Gitanshu1504/24DIT070_BDA_Practicals_2026_SHP